

TD4 : CREATION D'UN AGENT ET WORKFLOW LLM

UN CLIENT/SERVEUR MCP BASIQUE

Nous allons donc nous baser sur la notion de MCP pour travailler sur notre agent data scientist. Cette fois-ci, ce sera lui qui sera aux baguettes pour choisir quand et comment lancer les fonctions désignées précédemment. Pour cela, nous utiliserons fastMCP, une librairie pythonique de MCP : <https://github.com/jlowin/fastmcp> & <https://ai.pydantic.dev/mcp/fastmcp-client/#usage>

Le but est de créer un agent qui permettra de calculer de manière déterministe des calculs. Les LLM ne sont pas forcément bon pour réaliser cela car leur compréhension des nombres peut être limitée ou peu fiable.

Nous commençons par créer un fichier **mcp_server.py**.

1. Nous commençons par créer une instance FastMCP qui représentera notre serveur.
2. Nous créons ensuite 4 fonctions asynchrones représentant : **addition, soustraction, multiplication & division**. *Attention, il est important de respecter ce que l'on a expliqué lors de la partie théorique sur l'importance du typage et de la documentation*. Grâce à la documentation fastmcp & pydantic-ai, habillez les fonctions grâce au décorateur tool (`@mcp_calculator.tool()`, dans mon cas où mon serveur s'appelle mcp_calculator).

Nous disposons donc désormais d'un serveur avec 4 outils, nous allons donc créer le client (**mcp_client.py**) qui s'occupera d'interagir avec le serveur. Pour cela, nous nous utilisons Pydantic-AI (<https://ai.pydantic.dev/agents/>) qui nous permet de créer facilement un agent :

```
toolset = FastMCPToolset(mcp_calculator)

model = MistralModel("mistral-large-latest")

agent = Agent(
    model=model,
    toolsets=[toolset],
    system_prompt="You are a helpful calculator assistant. Use the available math tools to solve calculations step by step.",
)
```

Pour appeler l'agent, il est possible entre autres (voir documentation) d'utiliser la commande suivante : `result = await agent.run("What is 15 plus 27? and then multiply by 95!")`

Grâce à la documentation (<https://ai.pydantic.dev/agents/>), créer une fonction que nous pouvons appeler en ligne de commande ([utilisez argparse pour pouvoir passer des arguments lorsque vous exécutez la fonction via le terminal](#)), qui permette de requêter notre agent.

Pour finir, afin de pouvoir voir les différents appels d'outils réalisés, il faut que la fonction affiche non seulement la réponse de l'agent mais aussi les différents outils. Pour cela, je vous conseille d'utiliser la méthode de la classe AgentRunResult `.new_messages()`. [Voici un exemple de réponse attendue](#) :

```
Query: What is 15 plus 27? And then subtract 5, and finally divide by 0.75, finally add 1995

Tool Call History:
Tool: add
  Args: {"a": 15, "b": 27}
  Result: {'result': 42.0}
Tool: subtract
  Args: {"a": 42, "b": 5}
  Result: {'result': 37.0}
Tool: divide
  Args: {"a": 37, "b": 0.75}
  Result: {'result': 49.333333333333336}
Tool: add
  Args: {"a": 49.333333333333336, "b": 1995}
  Result: {'result': 2044.3333333333333}

Final Answer:
The final result is 2044.33 (rounded to two decimal places).
```

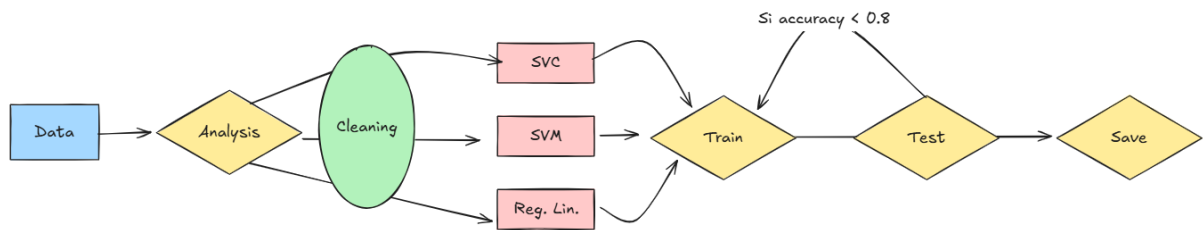
LE WORKFLOW AI

Votre rôle est de créer votre « remplaçant » ! Nous allons créer un agent Data Scientist : son rôle sera de s'occuper de trouver l'algorithme adéquat après avoir interagit avec des données spécifiées. **Le processus se déroule en plusieurs étapes fonctionnelles :**

- *Evaluer les données et la représentation de celles-ci.*
- *Potentiellement, nettoyer les données*
- *Choisir l'algorithme le plus adapté*
- *Entraîner le modèle le plus adapté*
- *Evaluer le modèle entraîné*
- *Sauvegarder le modèle le plus adapté*

Nous travaillerons comme les TD précédents avec différents scripts.

LE WORKFLOW DATA SCIENTIST



Le pipeline se décompose en **8 étapes clés**, chacune correspondant à une fonction ou classe spécifique du projet.

Chaque brique correspond à un fichier .py différent.

1 Chargement et préparation des données

a. Écrivez une fonction `load_dataset` permettant de charger un dataset soit depuis **scikit-learn**, soit depuis un **fichier externe (CSV ou Excel)**. La fonction doit prendre en entrée les paramètres suivants :

- `path` : chemin vers un fichier CSV ou Excel,
- `dataset_name` : nom d'un dataset scikit-learn,
- `target` : nom de la variable cible (obligatoire pour un fichier externe),
- `test_size` et `random_state` pour le découpage train/test.

La fonction devra retourner `X_train`, `X_test`, `y_train`, `y_test` ainsi que le type de tâche (classification ou regression).

b. Si un `dataset_name` est fourni, chargez automatiquement l'un des datasets suivants :

- **Classification** : `iris`, `wine`, `breast_cancer`
- **Régression** : `diabetes`, `california_housing`

Vous utiliserez les loaders de `sklearn.datasets` avec l'option `as_frame=True`.

c. Si un `path` est fourni, chargez le dataset à l'aide de **pandas** (`read_csv` ou `read_excel`). Vérifiez que la colonne cible existe bien, puis séparez les variables explicatives `X` de la variable cible `y`.

d. Déterminez automatiquement le type de tâche :

- si la variable cible est de type object ou possède moins de 20 valeurs uniques
→ **classification**,
- sinon → **régression**.

e. Appliquez un découpage **train/test** à l'aide de `train_test_split`.

Pour une tâche de classification, vous utiliserez l'argument `stratify=y` afin de conserver la distribution des classes dans les ensembles d'entraînement et de test.

f. Testez votre fonction sur au moins un dataset scikit-learn et un dataset externe, puis vérifiez les dimensions des jeux d'entraînement et de test ainsi que le type de tâche retourné

2. Nettoyage de la cible

a. Écrivez une fonction `clean_dataframe` permettant de nettoyer un **DataFrame complet** (hors variable cible). Cette fonction prendra en entrée le DataFrame, le type de jeu de données ("train" ou "test") ainsi que le nom éventuel de la colonne cible à exclure du traitement.

b. Pour les **variables catégorielles**, appliquez un **Label Encoding** à l'aide de `LabelEncoder` de `sklearn.preprocessing`.

- Lors du passage sur le jeu d'entraînement, les encodeurs devront être **appris** et stockés.
- Lors du passage sur le jeu de test, vous devrez **réutiliser les encodeurs appris** sur le jeu d'entraînement.
Une erreur devra être levée si un encodeur est manquant.

c. Pour les **variables numériques**, appliquez une **standardisation** à l'aide de `StandardScaler`.

Comme précédemment, les scalers devront être ajustés uniquement sur le jeu d'entraînement, puis réutilisés pour transformer le jeu de test.

d. Écrivez une seconde fonction `clean_target` dédiée au **nettoyage de la variable cible** :

- pour une tâche de **classification**, appliquez un `Label Encoding` ;
- pour une tâche de **régression**, conservez la variable cible telle quelle (conversion explicite en float).

f. Comme pour les variables explicatives, assurez-vous que l'encodage de la variable cible est appris uniquement sur le jeu d'entraînement et réutilisé sur le jeu de test.

g. Appliquez vos fonctions de nettoyage sur les jeux `X_train`, `X_test`, `y_train` et `y_test`, puis vérifiez que les dimensions des données restent inchangées après transformation.

3. Analyse exploratoire et LLM

- a. Créez une classe `Analysis` permettant de réaliser une **analyse exploratoire automatique** d'un jeu de données. Le constructeur devra prendre en entrée la matrice de variables explicatives `X`, la variable cible `y` (optionnelle) ainsi que le type de tâche (classification ou regression, optionnel).
- b. Si le type de tâche n'est pas fourni, inférez-le automatiquement à partir de la variable cible :
 - cible catégorielle ou numérique avec peu de valeurs uniques → **classification**,
 - cible numérique avec beaucoup de valeurs uniques → **régression**.
Affichez le type de tâche détecté lors de l'instanciation de la classe.
- c. Implémentez une méthode `basic_stats` calculant des **statistiques descriptives** (moyenne, écart-type, minimum, maximum) pour les variables numériques uniquement. Les résultats devront être stockés dans un dictionnaire interne `summary`.
- d. Implémentez une méthode `variable_types` permettant de compter le nombre de variables **numériques**, **catégorielles**, **booléennes**, ainsi que le nombre de lignes et de colonnes du dataset. Ces informations devront également être ajoutées au résumé.
- e. Implémentez une méthode `target_summary` dédiée à l'analyse de la variable cible :
 - pour une tâche de **classification**, calculez la distribution des classes (normalisée) ;
 - pour une tâche de **régression**, calculez des statistiques descriptives (moyenne, écart-type, minimum, maximum).
- f. Créez une méthode `compile_for_llm` permettant de transformer l'ensemble des informations collectées en un **rapport textuel structuré pour le LLM**. Le rapport devra inclure les caractéristiques du dataset, les statistiques numériques et un résumé de la variable cible.
- g. À l'aide de la librairie `litellm`, implémentez une méthode `generate_llm_insights` qui envoie ce rapport à un **LLM** afin d'obtenir un diagnostic automatique sur la qualité et

la nature des données. Le modèle devra uniquement fournir une analyse textuelle, sans générer de code.

h. Créer la méthode `__call__` afin de lancer l'ensemble du pipeline d'analyse (statistiques, types de variables, analyse de la cible et appel au LLM) en une seule instruction. Vérifiez que la sortie finale est un dictionnaire récapitulatif contenant toutes les informations collectées (statistiques, diagnostic du LLM, etc.)

4. Sélection du modèle

a. Créez un **registre de modèles** `MODEL_REGISTRY` associant des noms de modèles (sous forme de chaînes de caractères) aux classes correspondantes de **scikit-learn**.

```
MODEL_REGISTRY = {
    # Classification
    "LogisticRegression": LogisticRegression,
    "RandomForestClassifier": RandomForestClassifier,
    "ExtraTreesClassifier": ExtraTreesClassifier,
    "GradientBoostingClassifier": GradientBoostingClassifier,
    "HistGradientBoostingClassifier": HistGradientBoostingClassifier,
    "SVC": SVC,
    "KNeighborsClassifier": KNeighborsClassifier,
    "GaussianNB": GaussianNB,
    # Regression
    "LinearRegression": LinearRegression,
    "Ridge": Ridge,
    "Lasso": Lasso,
    "ElasticNet": ElasticNet,
    "RandomForestRegressor": RandomForestRegressor,
    "ExtraTreesRegressor": ExtraTreesRegressor,
    "GradientBoostingRegressor": GradientBoostingRegressor,
    "HistGradientBoostingRegressor": HistGradientBoostingRegressor,
    "SVR": SVR,
    "KNeighborsRegressor": KNeighborsRegressor,
}
```

b. Écrivez une fonction `select_model_by_name` qui prend en entrée le nom exact d'un modèle et retourne une **instance sklearn** correspondante. La fonction devra lever une erreur explicite si le nom du modèle n'existe pas dans le registre.

c. Créez une classe `ModelSelectorLLM` dont le rôle est de **sélectionner automatiquement un modèle** de machine learning à partir des **insights générés lors de l'analyse exploratoire**.

d. Le constructeur de la classe devra prendre en entrée un texte contenant les insights produits par le LLM lors de l'analyse du dataset.

e. Implémentez une méthode `suggest_model` qui envoie au LLM :

- le résumé et les insights du dataset,
- la liste des modèles disponibles dans le registre.

Le LLM devra répondre uniquement par le **nom exact** du modèle qu'il juge le plus approprié. Vérifiez que le modèle retourné appartient bien à la liste des options autorisées.

f. Stockez le nom du modèle sélectionné et implémentez une méthode `get_model_instance` permettant de récupérer une **instance sklearn** du modèle choisi à partir du registre.

g. Créez la méthode `__call__` afin de lancer l'ensemble du processus de sélection (appel au LLM + instanciation du modèle) en une seule instruction.

h. Testez votre pipeline complet sur un dataset :

chargement → nettoyage → analyse → sélection du modèle.

Vérifiez que le modèle retourné est cohérent avec le type de tâche (classification ou régression).

5. Suggestion d'hyperparamètres

a. Écrivez une fonction `suggest_hyperparameters` dont l'objectif est d'utiliser un **LLM** pour proposer un ensemble d'**hyperparamètres pertinents** à tester lors d'une recherche de type `GridSearchCV`.

b. La fonction prendra en entrée :

- une **instance de modèle sklearn** déjà sélectionnée,
- les **insights textuels** générés précédemment lors de l'analyse du dataset,
- un paramètre de température pour contrôler la génération du LLM.

c. À partir du nom du modèle (`model.__class__.__name__`), construisez un prompt demandant au LLM de renvoyer un **dictionnaire Python valide**, compatible avec

GridSearchCV, contenant uniquement les hyperparamètres les plus importants à explorer (sans être trop exhaustif).

d. Appelez le LLM à l'aide de `litellm.completion` et récupérez sa réponse. Nettoyez la sortie afin de supprimer d'éventuels blocs de code Markdown (python ...).

e. Transformez la réponse textuelle du LLM en objet Python à l'aide de `eval`, puis vérifiez que le résultat obtenu est bien un dictionnaire. Une erreur explicite devra être levée si le parsing échoue ou si le format est incorrect.

f. Testez votre fonction sur plusieurs modèles (par exemple un modèle linéaire et un modèle d'arbres) et vérifiez que les hyperparamètres proposés sont cohérents avec le type de modèle sélectionné

6. Entraînement du modèle

a. Écrivez une fonction `train_model` permettant d'entraîner un modèle de **scikit-learn** à partir des données d'entraînement.

b. La fonction devra prendre en entrée :

- une **instance de modèle sklearn**,
- les données `X_train` et `y_train`,
- un dictionnaire optionnel d'hyperparamètres `param_grid`,
- une métrique de scoring optionnelle,
- un niveau de verbosité.

c. Si un `param_grid` est fourni, entraînez le modèle à l'aide de **GridSearchCV** avec une validation croisée à `cv=3`.

d. Récupérez le **meilleur modèle** (`best_estimator_`) ainsi que les informations importantes issues de la recherche d'hyperparamètres :

- meilleurs paramètres,
- meilleur score de validation croisée,
- résultats complets de la validation croisée.

e. Si aucun `param_grid` n'est fourni, entraînez simplement le modèle en appelant la méthode `fit` sur les données d'entraînement.

f. La fonction devra retourner le **modèle entraîné** ainsi qu'un dictionnaire contenant les informations liées à GridSearchCV (ou un dictionnaire vide si aucune recherche d'hyperparamètres n'a été effectuée).

7. Évaluation du modèle

7. a. Créez une classe `TrainingEvaluatorLLM` dont l'objectif est d'**évaluer automatiquement les performances d'un modèle entraîné**, puis d'en proposer une **interprétation à l'aide d'un LLM**.
 - b. Le constructeur de la classe devra prendre en entrée :
 - le **modèle sklearn entraîné**,
 - les données de test `X_test` et `y_test`,
 - des insights optionnels sur le dataset (issus de l'analyse exploratoire).
 - c. Implémentez une méthode interne `_evaluate_model` chargée de calculer les **métriques d'évaluation adaptées au type de modèle** :
 - pour une tâche de **classification** : accuracy, F1-score pondéré, classification report et matrice de confusion ;
 - pour une tâche de **régression** : MSE, MAE et R^2 .
- Les résultats devront être regroupés dans un dictionnaire structuré.
- d. Implémentez une méthode `evaluate` qui :
 1. calcule les métriques d'évaluation sur le jeu de test,
 2. transmet le rapport obtenu à un **LLM** afin qu'il juge si l'entraînement du modèle est efficace ou non.
- Le LLM devra produire une courte synthèse textuelle ainsi qu'une décision finale (efficace ou inefficace) en tenant compte des risques de sous-apprentissage, sur-apprentissage, déséquilibre des classes, biais ou variance élevée.
- e. À partir de la réponse du LLM (**presence du mot efficace vs inefficace**), déduisez automatiquement un booléen indiquant si l'entraînement est jugé efficace. Stockez à la fois le résumé textuel et la décision dans les attributs de la classe.
 - f. Créez la méthode `__call__` afin de pouvoir lancer l'évaluation complète du modèle en une seule instruction.
 - g. Appliquez cette classe à votre modèle final et analysez la décision retournée par le LLM.

8. Sauvegarde du modèle

- a. Écrivez une fonction `save_model` permettant de **sauvegarder un modèle sklearn entraîné** sur le disque à l'aide de la librairie `joblib`.
- b. La fonction devra prendre en entrée :
 - le modèle entraîné,
 - le nom du fichier de sauvegarde (par exemple `model.joblib`).
- c. Assurez-vous que le modèle est enregistré dans un dossier nommé `storage/`. Si ce dossier n'existe pas, il devra être créé automatiquement.
- d. Sauvegardez le modèle à l'aide de `joblib.dump` et affichez un message confirmant l'emplacement du fichier sauvegardé.
- e. Testez la fonction en sauvegardant votre modèle final, puis vérifiez manuellement que le fichier est bien présent dans le dossier `storage`.

Flux global du workflow

Charger le dataset → 2. Nettoyer les données → 3. Analyser les données + LLM → 4. Sélectionner le modèle → 5. Déterminer les hyperparamètres → 6. Entraîner le modèle → 7. Évaluer les performances + LLM → 8. Sauvegarder le modèle si efficace.

9. a. Implémentez une fonction `full_ml_workflow` mettant en œuvre un **pipeline complet de machine learning**, depuis le chargement des données jusqu'à la sauvegarde du modèle final.
- b. Le pipeline devra accepter en entrée soit :
 - un chemin vers un dataset externe (CSV ou Excel),
 - soit le nom d'un dataset `scikit-learn`.
- c. Commencez par charger le dataset à l'aide de la fonction de chargement implémentée précédemment, puis séparez les jeux d'entraînement et de test ainsi que le type de tâche.
- d. Nettoyez la variable cible pour les jeux d'entraînement et de test en utilisant la fonction dédiée. Vérifiez que les encodages appris sur le jeu d'entraînement sont correctement réutilisés.

e. Réalisez ensuite une **analyse exploratoire automatique** du jeu d'entraînement à l'aide de la classe Analysis, puis générez des insights via un **LLM**. Affichez le diagnostic textuel produit.

f. À partir des insights générés, sélectionnez automatiquement un **modèle de machine learning** en utilisant la classe ModelSelectorLLM. Affichez le modèle retenu.

g. Utilisez ensuite un LLM pour suggérer un ensemble d'**hyperparamètres pertinents**, puis entraînez le modèle à l'aide de la fonction train_model.

h. Évaluez les performances du modèle entraîné sur le jeu de test à l'aide de la classe TrainingEvaluatorLLM. Récupérez à la fois le résumé textuel du LLM et la décision indiquant si l'entraînement est jugé efficace.

i. Si l'entraînement est considéré comme efficace, sauvegardez le modèle dans le dossier storage/.

Dans le cas contraire, interrompez le pipeline et affichez le diagnostic fourni par le LLM.

j. Vérifiez que le pipeline s'exécute correctement de bout en bout

Si vous avez le temps, vous pouvez visualisez le workflow via une interface streamlit.